

Etapa	Tiempo (aprox.)	Narrador (Voz en off)	Código en Pantalla (Foco en...)	Requisito Demostrado
I. Introducción y Modularidad	0:00 - 0:30	Iniciamos la revisión del código en agenda_contactos_inicial.py . El programa fue diseñado con un enfoque modular, tal como se sugiere en la guía, usando funciones para cada tarea principal.	VS Code abierto. Scroll rápido por la lista de funciones (cargar_contactos , agregar_contacto , main , etc.).	Función: Diseñar un programa con menú y funciones.

ACCIÓN EN PANTALLA: Mostrar el inicio del archivo agenda_contactos_inicial.py.

```
# --- agenda_contactos_completo_corregido.py ---  
  
import os  
import time  
import datetime  
  
# --- Constantes ---  
FILENAME = "contactos.txt"  
SEPARATOR = ";
```

Empezamos definiendo las constantes. Utilizamos FILENAME y SEPARATOR **[PARA QUÉ]** para centralizar y hacer más legible la ruta del archivo y el delimitador. Esto nos permite cambiar el nombre del archivo o el separador sin tener que modificarlo en todo el código. Noten que solo usamos módulos de la biblioteca estándar de Python, **[POR QUÉ]** lo que garantiza que el proyecto no necesite dependencias externas ni instalaciones adicionales

II. Manejo de Archivos y Persistencia	0:30 - 1:15	El primer objetivo era gestionar la persistencia de datos. Esto se logra con cargar_contactos y guardar_contactos .	Se selecciona cargar_contactos() . Mostrar el uso de open(FILENAME, 'r') y cómo se lee y separa la línea (linea.strip().split(SEPARATOR)). Luego, un vistazo a guardar_contactos() (SEPARATOR.join).	Objetivo: Leer y escribir información en archivos de texto (.txt).
---------------------------------------	-------------	---	---	--

ACCIÓN EN PANTALLA: Scroll a la función cargar_contactos(). Foco en el with open y el try/except.

```
def cargar_contactos():  
    """Carga los contactos desde el archivo de texto."""  
    contactos = []  
    try:  
        with open(FILENAME, 'r') as f:  
            for linea in f:  
                if linea.strip():  
                    partes = linea.strip().split(SEPARATOR)  
                    # ... (resto de la lógica) ...  
    except FileNotFoundError:  
        print(f"Advertencia: El archivo {FILENAME} no existía. Se ha creado uno nuevo.")  
        open(FILENAME, 'w').close()  
    return contactos
```

 **Explicación (Narración)** Esta función es la base de la persistencia de datos. Utilizamos el bloque with open **[POR QUÉ]** para asegurar que el archivo se cierre automáticamente, incluso si ocurre un error. **[PARA QUÉ]** El uso del try/except FileNotFoundError es vital para el manejo de errores: si el archivo contactos.txt no existe al inicio, evitamos que el programa falle, y en su lugar, creamos uno nuevo vacío. De esta manera, el programa es robusto y se autoinicia desde la primera ejecución."

III. Gestión de Errores (try/except)	1:15 - 1:50	Para el manejo de errores, una implementación clave es el uso de try/except al inicio del programa. Si el archivo contactos.txt no existe, la aplicación lo crea automáticamente y continúa, en lugar de detenerse.	Se enfoca en el bloque try/except FileNotFoundError dentro de cargar_contactos() .	Objetivo: Manejar errores comunes (archivo no existente) usando try/except.
---------------------------------------	-------------	---	--	---

ACCIÓN EN PANTALLA: Scroll a la función guardar_contactos(). Foco en SEPARATOR.join.

```
def guardar_contactos(contactos):
    """Guarda la lista de diccionarios de contactos en el archivo."""
    with open(FILENAME, 'w') as f:
        for contacto in contactos:
            linea = SEPARATOR.join([
                # ... campos del diccionario ...
            ])
            f.write(linea + '\n')
```

 **Explicación (Narración):** "La función guardar_contactos se encarga de serializar la lista de diccionarios de vuelta al archivo de texto. **[POR QUÉ]** Usamos el método SEPARATOR.join para convertir de manera eficiente el diccionario de Python a una cadena de texto. **[PARA QUÉ]** Esto garantiza que el formato de persistencia sea consistente, usando el punto y coma como delimitador en cada línea, tal como se especificó en el diseño

IV. Validaciones Robustas (Mejora al Requisito)	1:50 - 2:30	Además de la validación básica solicitada (no dejar todos los campos vacíos), implementamos validaciones mejoradas dentro de agregar_contacto() .	Mostrar la validación de <i>nombre obligatorio</i> (if not nombre.strip(): return). Mostrar la validación <i>medio de contacto requerido</i> (if not telefono.strip() and not correo.strip(): return).	Objetivo: Validar que el usuario no deje todos los campos vacíos (y mostrar las mejoras de validación).
---	-------------	--	---	---

ACCIÓN EN PANTALLA: Scroll a la función agregar_contacto(). Foco en las validaciones de nombre y telefono/correo.

```
nombre = input("Nombre completo (obligatorio): ")
if not nombre.strip():
    print("\nError: El nombre es un campo obligatorio. Operación cancelada.")
    return

# ... (código intermedio) ...

if not telefono.strip() and not correo.strip():
    print("\nError: Debes proporcionar al menos un teléfono o un correo electrónico.")
    print("Operación cancelada.")
    return
```

 **Explicación (Narración):** "Aquí es donde se aplican las reglas de negocio para la integridad de los datos. **[POR QUÉ]** En la validación del medio de contacto, utilizamos un operador and: la condición if not telefono.strip() and not correo.strip(): solo se cumple si **ambos** campos están vacíos. **[PARA QUÉ]** Esto asegura que cada contacto tenga al menos una forma de ser localizado, haciendo la agenda útil."

V. Validación de Formato Estricto	2:30 - 3:00	La consistencia de datos es vital. La función pedir_fecha_valida() se encarga de que la fecha de reunión siempre tenga el formato DD MM AAAA. Usamos el módulo estándar datetime de Python para esta verificación estricta.	Se enfoca en pedir_fecha_valida() y el uso de datetime.datetime.strptime.	Funcionalidad: Implementación estricta de la Fecha de próxima reunión.
---	-------------	--	---	--

Etaa V. Validación de Formato: pedir_fecha_valida()

ACCIÓN EN PANTALLA: Scroll a la función pedir_fecha_valida(). Foco en el while True y el try/except ValueError.

```
def pedir_fecha_valida():
    """Pide al usuario una fecha en formato DD MM AAAA y la valida."""
    while True:
        fecha_str = input("Introduce la fecha (formato DD MM AAAA): ")
        try:
            datetime.datetime.strptime(fecha_str.strip(), '%d %m %Y')
            return fecha_str.strip()
        except ValueError:
            print("Error: Formato incorrecto. Por favor, use DD MM AAAA...")
```

 **Explicación (Narración)** Para la consistencia de la agenda, necesitamos un formato de fecha estricto. **[POR QUÉ]** Esta función utiliza un bucle while True junto a try/except para validar el formato. Si la librería datetime falla al intentar convertir la cadena a nuestro formato exacto (%d %m %Y), lanza un ValueError. **[PARA QUÉ]** Capturamos ese error para mostrar un mensaje útil y forzamos al usuario a corregir la entrada, garantizando datos limpios en el archivo.

VI. Corrección de Lógica (actualizar _contacto)	3:00 - 3:30	Finalmente, la función actualizar_contacto cumple con el requisito de modificar la fecha. Noten que la implementación incluye una búsqueda por nombre, y si no se encuentra, se informa al usuario. Esta es una corrección importante para evitar errores lógicos si el contacto no existe.	Mostrar el bloque if <u>contacto a actualizar:</u> y el else: que imprime el mensaje de "no encontrado".	Funcionalidad: Registrar/Actualizar fecha (y manejo del caso de contacto no existente).
---	-------------	---	---	---

ACCIÓN EN PANTALLA: Scroll a la función listar_contactos(). Foco en la línea de sort.

```
def listar_contactos(contactos):
    # ...
    else:
        # Ordenamos los contactos por nombre
        contactos.sort(key=lambda c: c["nombre"].lower())
    # ...
```

 **Explicación (Narración)** Para mejorar la usabilidad, la lista se presenta siempre ordenada. **[POR QUÉ]** Usamos el método .sort() y el parámetro key para definir cómo debe ordenar Python una lista de diccionarios. La clave es la función lambda c: c["nombre"].lower(). Esta función toma cada contacto, extrae el nombre, y lo convierte a minúsculas. **[PARA QUÉ]** Esto permite ordenar la lista por nombre y garantiza que la clasificación sea insensible a mayúsculas, por lo que 'Ana' y 'ana' se ordenarán correctamente."

En resumen, esta línea de código permite ordenar una lista de diccionarios por el valor de su campo 'nombre', asegurando un orden alfabético coherente.

Etapa VII. Lógica y Corrección de Errores: actualizar_contacto()

ACCIÓN EN PANTALLA: Scroll a la función actualizar_contacto(). Foco en la búsqueda y el bloque else.

```
contacto_a_actualizar = None
for contacto in contactos:
    if contacto["nombre"].lower() == nombre_buscado.lower():
        contacto_a_actualizar = contacto
        break

if contacto_a_actualizar:
    # ... lógica de actualización ...
else:
    print(f"\nNo se encontró ningún contacto con el nombre '{nombre_buscado}'.")
```



Explicación (Narración): "Esta función demuestra una lógica de control de flujo robusta. **[POR QUÉ]** Implementamos una búsqueda que ignora mayúsculas y minúsculas. **[PARA QUÉ]** Lo más importante: si la búsqueda no encuentra una coincidencia, el programa no intenta actualizar nada. El bloque else se activa para informar al usuario de que el contacto no existe, lo cual es una corrección importante para evitar fallos de ejecución y asegurar una buena experiencia de usuario."

Etapa VIII. Control del Flujo: main()

ACCIÓN EN PANTALLA: Scroll a la función main() y el bloque final.

```
def main():
    contactos = cargar_contactos()
    while True:
        # ... (código del menú) ...

if __name__ == "__main__":
    main()
```



Explicación (Narración)

"Finalmente, la función main gestiona el bucle principal (while True) que repite el menú hasta que el usuario elige salir. **[POR QUÉ]** El bloque `if __name__ == '__main__':` es el punto de entrada estándar en Python que asegura que la función main se ejecute solo cuando el script es llamado directamente."

Con la revisión técnica completada, ahora pasaremos al terminal para ver estas funciones y validaciones en acción.